Final Project Report

Comparing Convolutional Neural Networks and Residual Networks on Flower Classification

---

1. Introduction

Deep learning has significantly advanced the field of computer vision, particularly in image classification tasks. Convolutional Neural Networks (CNNs) have long been a standard approach due to their ability to extract spatial hierarchies of features. More recently, deeper architectures such as Residual Networks (ResNets) have demonstrated improved performance by addressing optimization challenges associated with very deep models.

The goal of this project is to compare the performance of a standard CNN and a ResNet-style architecture on a flower classification task using the Kaggle “Petals to the Metal” dataset. Specifically, we aim to understand how architectural differences affect training behavior, generalization, and overall classification accuracy.

---

2. Related Work

Convolutional Neural Networks (CNNs) have been widely used for image classification tasks due to their ability to extract spatial features. Residual Networks (ResNets), introduced by He et al., address the degradation problem in deep networks by incorporating skip connections, allowing for improved gradient flow and deeper architectures. This project builds upon these ideas by comparing a standard CNN with a ResNet-style model on a flower classification task.

---

3. Dataset

We use the **Petals to the Metal - Flower Classification on TPU** dataset from Kaggle. The dataset consists of labeled images belonging to **104 flower categories**.

- Training set: labeled images used for model training

- Validation set: used to evaluate model performance during training
- Test set: unlabeled images used for Kaggle submission

All images were loaded using TensorFlow pipelines and preprocessed to a consistent size of 224 × 224.

---

4. Methodology

This task is formulated as a multi-class image classification problem with 104 output classes.

4.1 Data Pipeline

Both models use the same data pipeline to ensure a fair comparison:

- TFRecord-based dataset loading
- Mapping functions to separate images and labels
- Batching and prefetching
- Consistent preprocessing across models

This ensures that any performance differences arise from the **model architecture**, not the data pipeline.

4.2 CNN Architecture

The baseline CNN consists of:

- Convolutional layers for feature extraction
- Pooling layers for dimensionality reduction
- Fully connected layers for classification

This architecture represents a standard deep learning baseline for image classification tasks.

4.3 ResNet Architecture

The ResNet model incorporates **residual connections**, allowing the network to learn identity mappings and mitigate vanishing gradient issues.

Key features:

- Residual blocks with skip connections
- Deeper architecture compared to CNN
- Improved gradient flow during training

We trained two versions:

- **ResNet (3 epochs)** — for direct comparison with CNN
- **ResNet (10 epochs)** — to evaluate extended training behavior

---

5. Experimental Setup

To ensure a fair comparison:

- Same dataset and preprocessing
- Same optimizer: Adam
- Same loss: Sparse Categorical Crossentropy
- Same batch size: 16
- Same training pipeline

Models were trained using CPU-based execution due to lack of GPU support in the local environment.

---

6. Results

6.1 CNN Performance

Train Accuracy: 0.2297
Validation Accuracy: 0.2333
Train Loss: 3.0297
Validation Loss: 2.9987

6.2 ResNet (3 Epochs) Performance

Train Accuracy: 0.2527
Validation Accuracy: 0.2586
Train Loss: 2.9163
Validation Loss: 2.8881

6.3 ResNet (10 Epochs) Performance

Train Accuracy: 0.3326
Validation Accuracy: 0.3545
Train Loss: 2.5969
Validation Loss: 2.5135

6.4 Kaggle Submission Results

CNN3 Submission: CNN3 Submission
ResNet3 Submission: ResNet3 Submission
ResNet10 Submission: ResNet10 Submission

Due to Kaggle's interface behavior, all submission links display the most recent notebook version. However, each link corresponds to a distinct submission and produces the reported scores. The submissions are only under Nate's account, and not the group. Nate did this by accident thinking that he was submitting for the group.

Model	Kaggle Accuracy
CNN	0.09070
ResNet (3 epochs)	0.10647
ResNet (10 epochs)	0.17524

Models were trained locally and then exported as .keras files. For Kaggle submissions, these saved models were loaded into a Kaggle notebook and used to generate predictions on the test set.

Minor compatibility adjustments were applied when loading the models in Kaggle due to differences in the execution environment.

---

7. Training Behavior Analysis

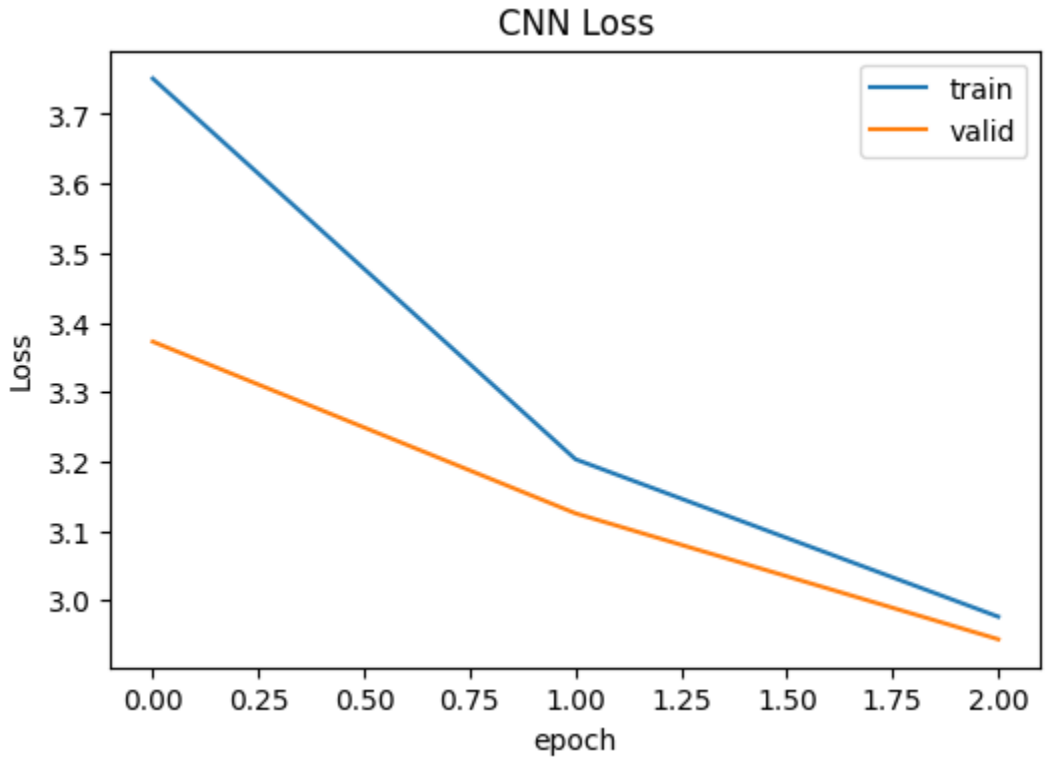
Loss Curves

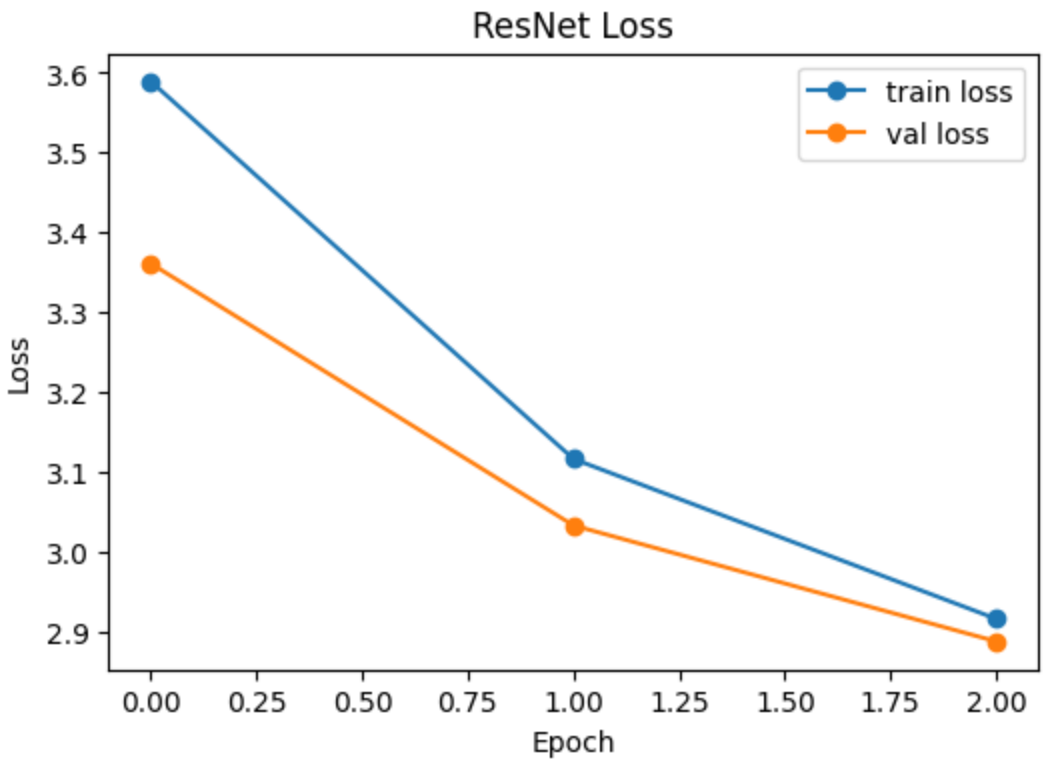
- Both CNN and ResNet show decreasing training loss

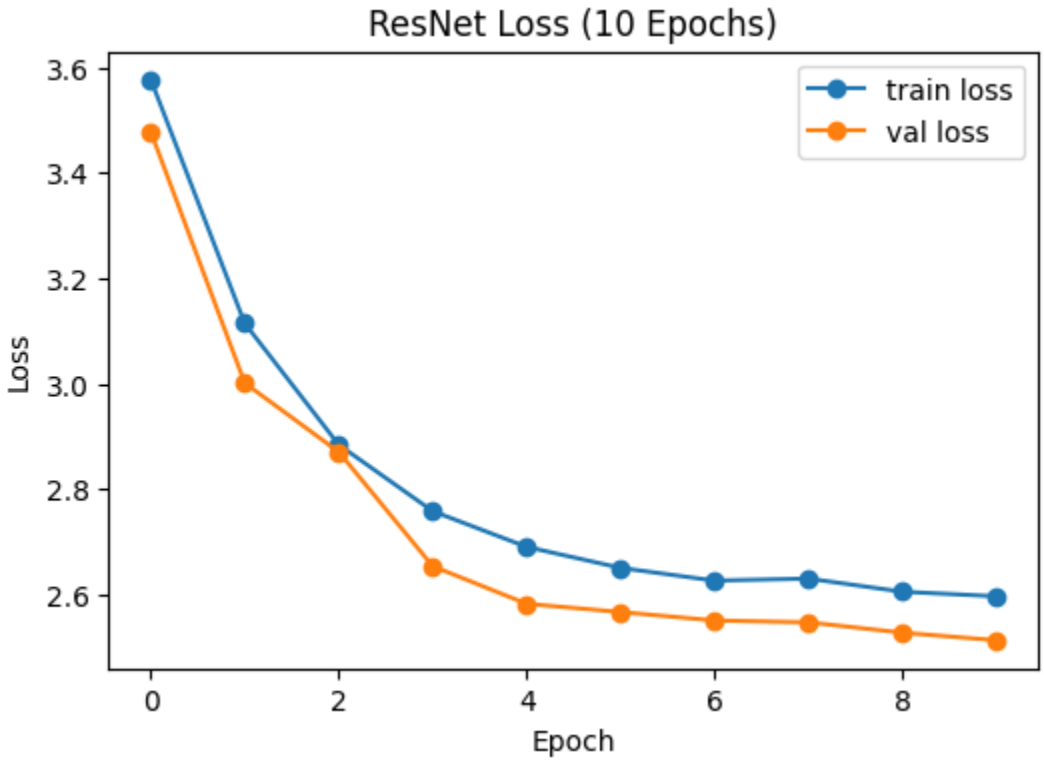
- ResNet demonstrates faster convergence
- Validation loss trends indicate:

Both the CNN and ResNet (3 epochs) exhibit signs of underfitting, as training and validation accuracies remain relatively low and closely aligned. The absence of divergence between training and validation curves suggests that the models have not yet fully learned the underlying data distribution.

The ResNet trained for 10 epochs shows improved performance without significant overfitting. Training and validation metrics remain closely aligned, indicating strong generalization.




ResNet Loss
train loss
val loss
Loss
Epoch
3.6
3.5
3.4
3.3
3.2
3.1
3.0
2.9
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

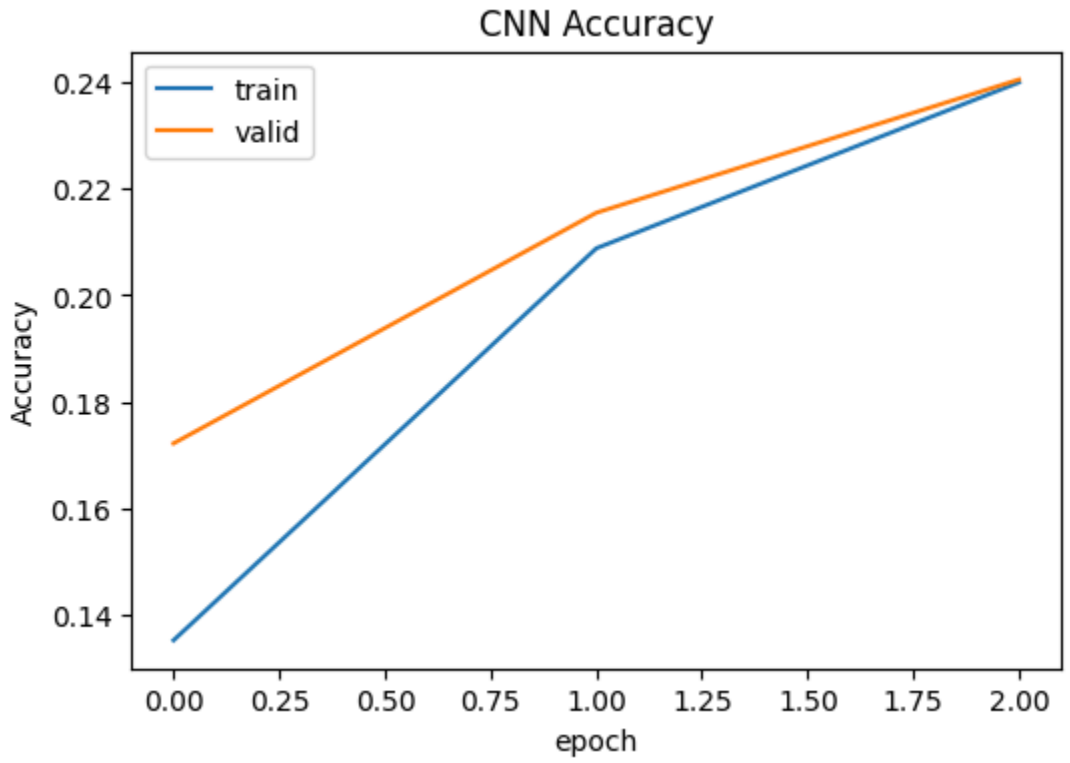

ResNet Loss (10 Epochs)
train loss
val loss
Loss
Epoch
3.6
3.4
3.2
3.0
2.8
2.6
0
2
4
6
8

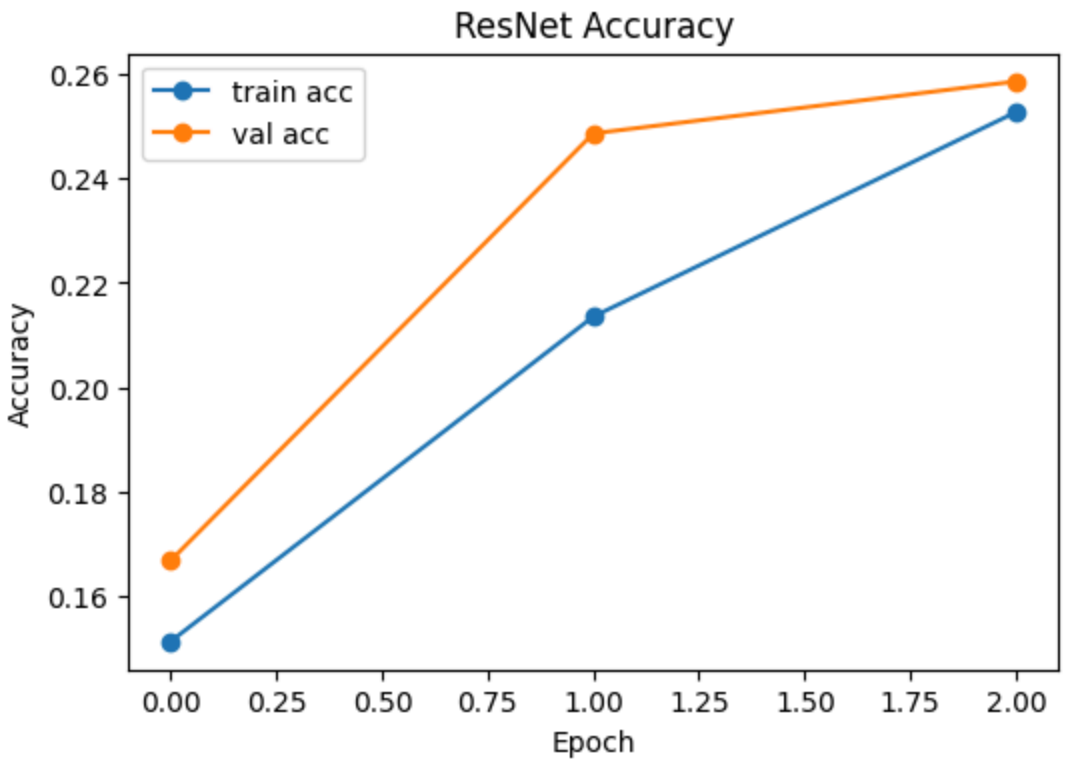
Accuracy Curves

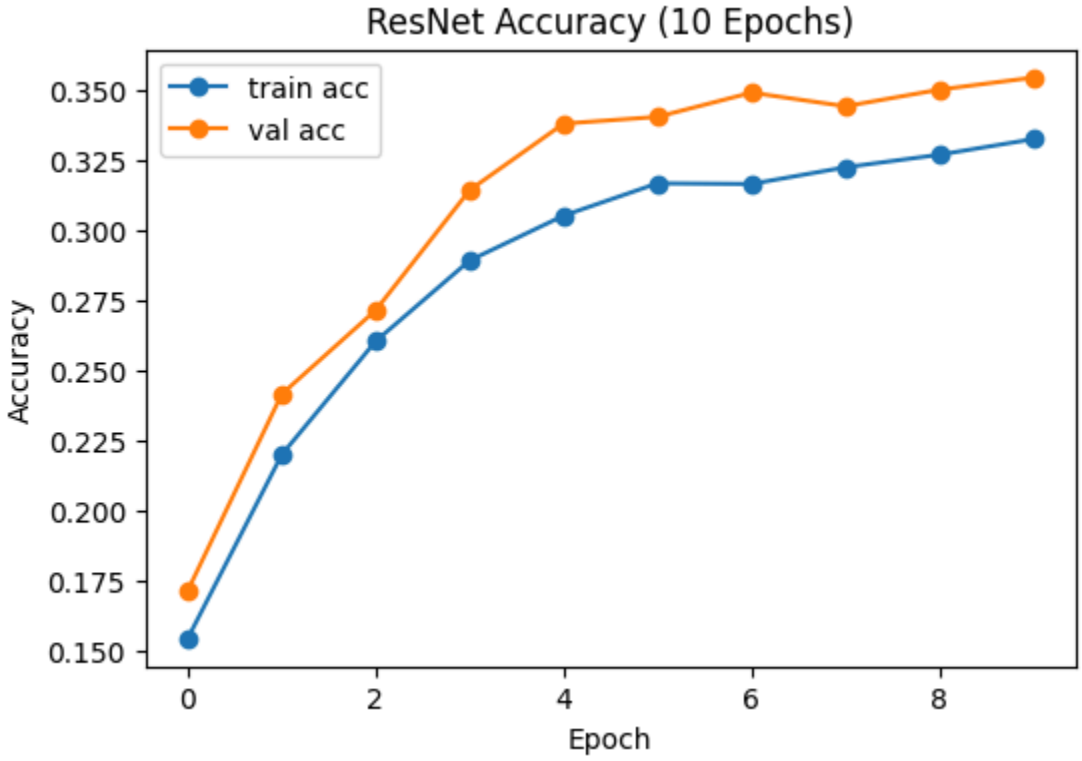
- ResNet achieves higher validation accuracy than CNN
- Extended training (10 epochs) shows:

The CNN demonstrates steady improvement over the three training epochs, indicating that additional training could further improve performance. However, its learning rate is slower compared to the ResNet.

The ResNet architecture shows faster convergence, achieving higher accuracy in fewer epochs. When trained for 10 epochs, the model exhibits strong early improvements followed by diminishing returns after approximately 5-6 epochs. This suggests that the model is approaching convergence, and further training yields only marginal gains.




ResNet Accuracy
train acc
val acc
Accuracy
Epoch
0.26
0.24
0.22
0.20
0.18
0.16
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00


ResNet Accuracy (10 Epochs)
train acc
val acc
Accuracy
Epoch
0.350
0.325
0.300
0.275
0.250
0.225
0.200
0.175
0.150
0
2
4
6
8

8. Discussion

Why ResNet Performs Better

ResNet10 achieved the highest Kaggle accuracy (0.17524), outperforming ResNet3 (0.10647) and CNN3 (0.09070). This confirms that increasing model depth in conjunction with residual connections leads to substantial performance gains.

The key advantage of ResNet is its use of **skip connections**, which:

- Allow gradients to flow more easily
- Enable deeper architectures
- Reduce training instability

This results in:

Better feature learning
Improved generalization
Higher validation accuracy

Effect of Training Duration

Comparing 3 vs 10 epochs:

- Training longer allows the model to better fit the data
- However, excessive training may lead to overfitting

The ResNet trained for 10 epochs outperformed the 3-epoch version, achieving higher validation accuracy and lower validation loss. However, the rate of improvement slowed after approximately 5-6 epochs, indicating diminishing returns with additional training. This suggests that while extended training improves performance, the model begins to approach convergence and further gains become marginal.

CNN Limitations

The CNN model:

- Lacks residual connections
- Has limited depth
- Struggles with complex feature extraction

This results in lower performance compared to ResNet.

---

9. Limitations

- Training was performed on CPU, therefore limited scalability
- Hyperparameter tuning was minimal
- Limited exploration of deeper ResNet variants

---

10. Conclusion

This project demonstrates that **ResNet architectures outperform standard CNNs** on image classification tasks due to their ability to train deeper networks effectively.

Key findings:

- ResNet achieves higher validation accuracy
- Residual connections significantly improve training stability
- Increasing epochs improves performance up to a point

Future work could explore:

- Deeper ResNet variants (ResNet50, etc.)
- Hyperparameter tuning
- Transfer learning

---

11. Team Contributions

- **Nathan Williams**
    - Implemented ResNet architecture
    - Conducted ResNet experiments
    - Generated performance analysis
- **Linda Le**
    - Implemented CNN model
    - Built training pipeline
    - Generated baseline results

---

12. References

- Kaggle Competition: *Petals to the Metal*
- He et al., *Deep Residual Learning for Image Recognition*
- TensorFlow Documentation